

Deep Learning

Vazgen Mikayelyan



1 Attention Models

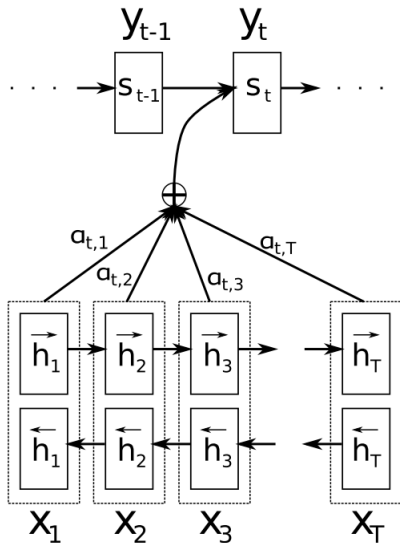
2 Transformers

- Encode each word in the sentence into a vector using RNNs.

- Encode each word in the sentence into a vector using RNNs.
- When decoding, perform a convex combination of these vectors, weighted by “attention weights”.

- Encode each word in the sentence into a vector using RNNs.
- When decoding, perform a convex combination of these vectors, weighted by “attention weights”.
- Use this combination in picking the next word.

Attention Model



Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

where

$$\alpha_{ij} = \frac{\exp \{e_{ij}\}}{\sum_{j=1}^{T_x} \exp \{e_{ij}\}}$$

Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

where

$$\alpha_{ij} = \frac{\exp \{e_{ij}\}}{\sum_{j=1}^{T_x} \exp \{e_{ij}\}}$$

and $e_{i,j}$ are calculated using alignment model

$$e_{ij} = a(s_{i-1}, h_j) = v_a^T \tanh(W_a s_{i-1} + U_a h_j)$$

Outline

1 Attention Models

2 Transformers

Problems with RNNs

- Sequential computation prevents parallelization.

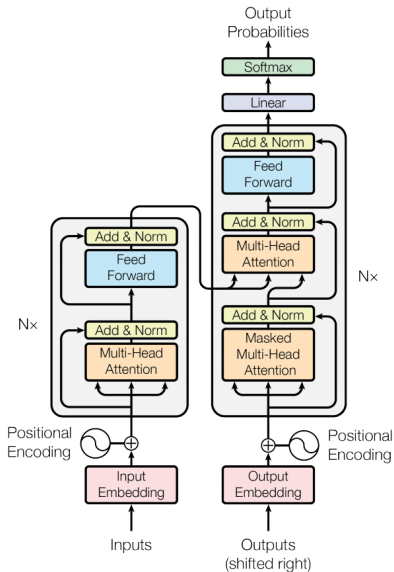
Problems with RNNs

- Sequential computation prevents parallelization.
- Despite GRUs and LSTMs, RNNs still need attention mechanism to deal with long range dependencies – path length for codependent computation between states grows with sequence.

Problems with RNNs

- Sequential computation prevents parallelization.
- Despite GRUs and LSTMs, RNNs still need attention mechanism to deal with long range dependencies – path length for codependent computation between states grows with sequence.
- But if attention gives us access to any state, maybe we don't need the RNN?

Transformer



Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.

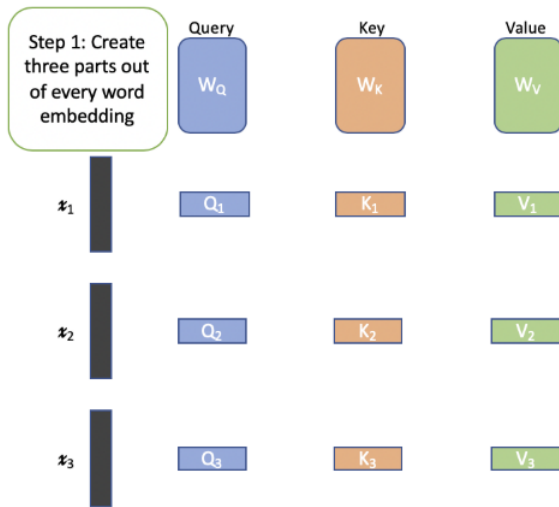
Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.
- Given an embedding x , it learns three separate smaller embeddings from it — query, key and value.

Self Attention Layer

- This layer aims to encode a word based on all other words in the sequence. It measures the encoding of the word against the encoding of another word and gives a new encoding.
- Given an embedding x , it learns three separate smaller embeddings from it — query, key and value.
- During the training phase, the W_q , W_k , and W_v matrices are learnt to get the query, key and value embeddings.

Self Attention Layer



Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .

Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.

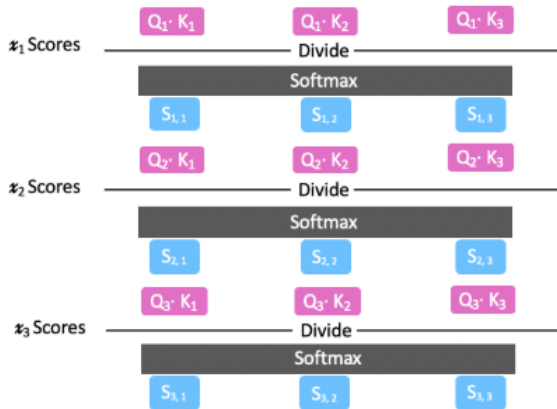
Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.
- Then x_1 will take all these scores and perform softmax.

Self Attention Layer

- Say x_1 wants to know its value with respect to x_2 . So it will 'query' x_2 .
- x_2 will provide the answer in the form of its own 'key', which can then be used to get a score representing how much it values x_1 by taking a dot product with the query. Since both have the same size, this will be a single number.
- Then x_1 will take all these scores and perform softmax.
- This step will be performed with every word.

Self Attention Layer



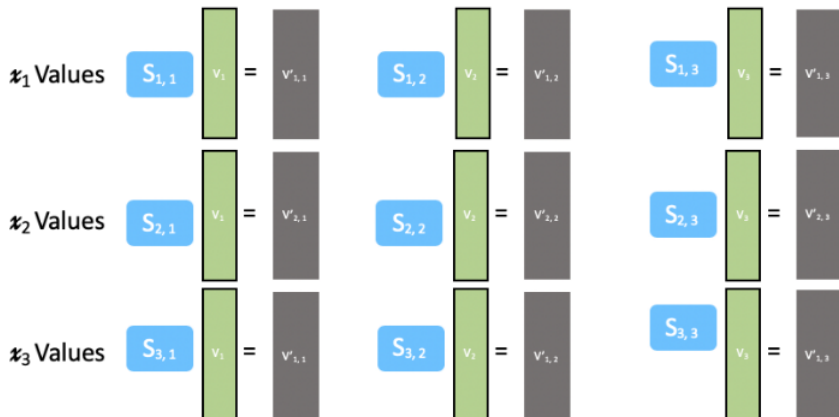
Self Attention Layer

- x_1 will now use this score and the 'value' of the corresponding word to get a new value of itself with respect to that word.

Self Attention Layer

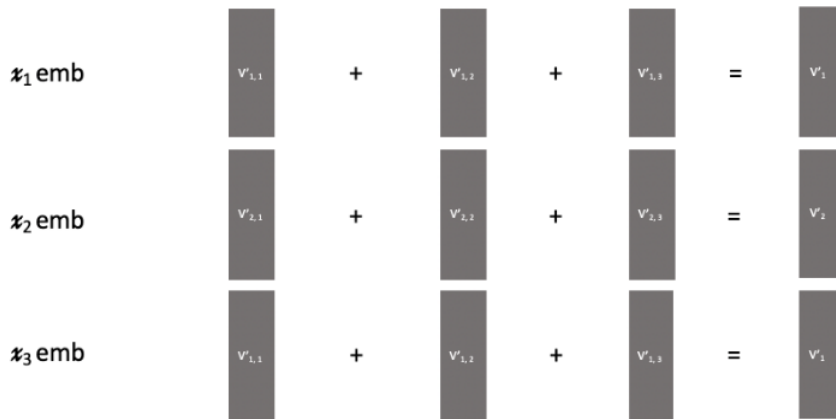
- x_1 will now use this score and the 'value' of the corresponding word to get a new value of itself with respect to that word.
- If the word is not relevant to x_1 then the score will be small and the corresponding value will be reduced a factor of that score and similarly the significant words will get their values bolstered by the score.

Self Attention Layer



Self Attention Layer

Finally, the word x_1 will create a new 'value' for itself by summing up the values received. This will be the new embedding of the word.



$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,
- weight of each value is computed by an inner product of query and corresponding key,

$$\text{Attention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

where

- inputs: a query q and a set of key-value (K-V) pairs to an output,
- query, keys, values and output are all vectors,
- output is a convex combination of values,
- weight of each value is computed by an inner product of query and corresponding key,
- queries and keys have the same dimensionality d_k , values have d_v .

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

- Problem: as d_k gets large, the variance of $q^T k$ increases, thus some values inside the softmax gets large, hence its gradients gets smaller.

When we have multiple queries q , we stack them in a matrix Q :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right) V.$$

- Problem: as d_k gets large, the variance of $q^T k$ increases, thus some values inside the softmax gets large, hence its gradients gets smaller.
- Solution:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-Head Attention Layer

Multihead attention assumes that all inputs and outputs have the same length d_{model} . If inputs hasn't length d_{model} , we pass it through one fully connected layer.

Multi-Head Attention Layer

Multihead attention assumes that all inputs and outputs have the same length d_{model} . If inputs hasn't length d_{model} , we pass it through one fully connected layer.

$$\text{Multihead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where

$$\text{head}_i = \text{Attention}(xW_i^Q, xW_i^K, xW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_{model} \times d_k}, W_i^K \in \mathbb{R}^{d_{model} \times d_k}, W_i^V \in \mathbb{R}^{d_{model} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_{model}}$$